Un lenguaje funcional puro sencillo (SFL) [2] y su intérprete

Eduardo Bonelli [3]

Departamento de Computación [4]
FCEyN
UBA

3 de octubre, 2006 [5]

Intérprete para un lenguaje funcional sencillo (SFL) 2

- Componentes de un intérprete 3
- Sintaxis concreta y abstracta de SFL 4
- Intérprete 5
- Extensiones 6
  1. binding local (`let`) 7
  2. procedimientos (`proc`) 8
  3. recursión (`letrec`) 9

Componentes de un intérprete

- Definición inductiva de 2
  - las expresiones sintácticamente correctas (*sintaxis concreta*) 3
  - la representación interna de las expresiones (*sintaxis abstracta*)
- Designar los valores expresados: subconjunto de expresiones a los que puede evaluar una expresión arbitraria 4
- Un entorno ("environment"): estructura de datos que liga variables con valores denotados 5
- Un entorno *inicial* 6
- Intérprete como función recursiva sobre el conjunto inductivo de las expresiones (ie. de la sintaxis abstracta) 7
- Salida del intérprete: valores expresados 8

Sintaxis Concreta de SFL

La sintaxis de SFL viene dada por la siguiente gramática BNF: 3

```
<program> ::= <expr>

<expr> ::= <number> | <identifier>
         | <primitive> ({<expr>*(,)})
         | if <expr> then <expr> else <expr>

<primitive> ::= + | - | * | add1 | sub1 | isZero
```

5

Ejemplos

Ejemplos de expresiones válidas: 2

```
3
n
+(3,n)
add1(add1(3,n),4)
if -(2,+(3,4)) then 2 else 3
+(if 2 then 3 else 3,7)
```

3

NOTA: `if` testea valores expresados 4

- **falso**: 0
- **verdadero**: cualquier valor expresado distinto de 0

5

Sintaxis abstracta

```
data Program = Pgm Exp 3

data Exp = LitExp Int |                4
           VarExp Symbol |
           IfExp Exp Exp Exp |
           PrimApp PrimOp [Exp]

data PrimOp = AddPrim | SubtractPrim | MultPrim | IncrPrim 5
              DecrPrim | ZeroTestPrim

type Symbol = String 6
```

Mapeo de Sintaxis Concreta a Abstracta

- El mapeo de sintaxis concreta a sintaxis abstracta se realiza en dos pasos: 3
  1. Análisis léxico ("lexer") 4
  2. Análisis sintáctico ("parser")
- La cátedra los proveerá de modo que puedan probar el intérprete 5
- Para el ejemplo: `let x=3 in *(2,x)` 6
- salida del lexer: `[TokenLet,TokenVar "x",TokenEq,TokenInt 3,TokenIn,TokenTimes, TokenOB, TokenInt 2, TokenComma, TokenVar "x", TokenCB]` 7
- salida del parser: `Pgm (LetExp ["x"] [LitExp 3] (PrimApp MultPrim [LitExp 2,VarExp "x"]))` 8

Mapeo de Sintaxis Concreta a Abstracta

- El mapeo de sintaxis concreta a sintaxis abstracta se realiza en dos pasos: 3
  1. Análisis léxico ("lexer") 4
  2. Análisis sintáctico ("parser")
- La cátedra los proveerá de modo que puedan probar el intérprete 5
- Para el ejemplo: `let x=3 in *(2,x)`
- salida del lexer: `[TokenLet,TokenVar "x",TokenEq,TokenInt 3,TokenIn,TokenTimes, TokenOB, TokenInt 2, TokenComma, TokenVar "x", TokenCB]`
- salida del parser: `Pgm (LetExp ["x"] [LitExp 3] (PrimApp MultPrim [LitExp 2,VarExp "x"]))`

Mapeo de Sintaxis Concreta a Abstracta 1

- El mapeo de sintaxis concreta a sintaxis abstracta se realiza en dos pasos: 2
  1. Análisis léxico ("lexer") 3
  2. Análisis sintáctico ("parser")
- La cátedra los proveerá de modo que puedan probar el intérprete 4
- Para el ejemplo: `let x=3 in *(2,x)`
- salida del lexer: `[TokenLet,TokenVar "x",TokenEq,TokenInt 3,TokenIn,TokenTimes, TokenOB, TokenInt 2, TokenComma, TokenVar "x", TokenCB]`
- salida del parser: `Pgm (LetExp ["x"] [LitExp 3] (PrimApp MultPrim [LitExp 2,VarExp "x"]))`

Valores

Valores expresados

Valores que puede arrojar la evaluación de una expresión. 3

- Ej. números, procedimientos

Valores denotados 4

Valores que se ligan a los identificadores (ie. a las variables). 5

- Ej. números, procedimientos 6
- Más adelante: referencias

NOTA: Con la simple descripción del conjunto de valores expresados y denotas ya se dice bastante sobre el lenguaje objeto. 7

Intérpete de programas

El valor expresado resultante de evaluar `+(3,4)` es claro. 2

- ¿Qué sucede con la expresión `+(x,5)`? 3
- ¿A qué valor expresado debe evaluar?

Es necesario contar con un entorno: estructura de datos que liga variables con valores denotados 4
Dado el entorno que liga el valor 2 a la variable, podemos responder que `+(x,5)` debe evaluar al valor 7.
Asumiremos que el intérprete comienza su ejecución contando con un entorno inicial.

Entornos

- Representación de entornos en Haskell 1

```
data Env = EmptyEnv |
           ExtendedEnvRecord [Symbol]
                             [ExpressedValue]
                             Env

data ExpressedValue = Nro {stripNro::Int}

-- extender entorno con nuevo rib
extendEnv::[Symbol]->[ExpressedValue]->Env->Env
extendEnv ids vals env = ExtendedEnvRecord ids vals env
```

2

Entornos

- Buscar el valor expresado asociado a una variable 1

```
applyEnv::Env->Symbol->ExpressedValue
applyEnv EmptyEnv sym =
        error ("applyEnv: Sin binding para "++ show sym)
applyEnv (ExtendedEnvRecord syms vals env) sym
     = case findInList syms sym of
       (Just n) -> vals!!n
       _ -> applyEnv env sym
```

2

```
-- buscar simbolo en lista de simbolos (retornar posicion)
findInList::[a]->a->Maybe Int
```

3

Intérpete de programas 2

```
-- interprete de programas
evalProgram (Pgm body) = evalExpr body initEnv

-- entorno inicial
initEnv = ExtendedEnvRecord ["i","v","x"]
                            [Nro 1, Nro 5, Nro 10]
                            EmptyEnv
```

3

Interpretando expresiones

```
evalExpr (LitExp n) env = Nro n
evalExpr (VarExp id) env = applyEnv env id
evalExpr (PrimApp prim rands) env =
      let args = map (\x->evalExpr x env) rands
        in applyPrimitive prim args
evalExpr (IfExp testExp trueExp falseExp) env =
     if isTrueValue (evalExpr testExp env)
        then evalExpr trueExp env
        else evalExpr falseExp env
```

2

```
isTrueValue::ExpressedValue->Bool
isTrueValue = not . (==0) . stripNro
```

3

Aplicando primitivas

```
applyPrimitive AddPrim args =                                   2
    Nro (stripNro (args!!0) + stripNro (args!!1))
applyPrimitive SubtractPrim args =
    Nro (stripNro (args!!0) - stripNro (args!!1))               3
applyPrimitive MultPrim args =
    Nro (stripNro (args!!0) * stripNro (args!!1))               4
applyPrimitive IncrPrim args = Nro (stripNro (args!!0) + 1)
applyPrimitive DecrPrim args = Nro (stripNro (args!!0) - 1)
applyPrimitive ZeroTestPrim  args =
    Nro (if stripNro (args!!0)==0 then 1 else 0)                5
```

Declaraciones locales a través de ejemplos 2

```
let x = 5,
    y = 6
in +(x,y)

-- dependencia entre declaraciones

let x = 3
  in let y = +(x,4)
         in *(x,y)

let x = 3,
    y = +(x,4)
in *(x,y)
```

3

Declaraciones locales a través de ejemplos 2

```
-- declaraciones internas enmascaran las externas 3

let x = 3
in let x = *(x,x)
   in +(x,x)
```

4

Declaraciones locales

```
let var1 = exp1,
    ...
    varn-1 = expn-1,
    varn = expn
  in cuerpo
```
3

Sintaxis concreta 4

`<expr> ::= let` $\{$`<ident> = <expr>`$\}^{*(,)}$ `in <expr>` 5

Región La región asociada con `var1`,...,`varn` es `cuerpo` 6

Evaluación

- cada `expi` (en ningún orden en particular!)
- ligar cada `vari` al valor de `expi`
- evaluar `cuerpo` y retornar su resultado como el resultado de toda la expresión.

7

Procedimientos a través de ejemplos 2

3

```
let f = proc (y,z) +(y,-(z,5))
  in (f 2 28)

(proc (y,z) +(y,-(z,5)) 2 28)

-- si bien no tenemos letrec, podemos hacer esto
let
  factAux = proc(next,x)
              if zero?(x) then 1
                          else (next next sub1(x))
in
  let fact = proc(x) (factAux factAux x)
  in (fact 3)
```

Procedimientos: ejemplos

Sintaxis concreta 3

```
<expr> ::= ... | proc ({<identifier>}*(,)) <expr>
           | (<expr> {<expr>}*)
```
4

Sintaxis abstracta 5

```
data Exp = LitExp Int |
           VarExp Symbol |
           IfExp Exp Exp Exp |
           PrimApp PrimOp [Exp] |
           LetExp [Symbol] [Exp] Exp |
           ProcExp [Symbol] Exp |
           AppExp Exp [Exp]
```
6

Extendiendo el intérprete (let) 3

```
evalExpr (LitExp n) env = Nro n
...
evalExpr (LetExp ids rands body) env =
        let args = map (\x-> evalExpr x env) rands
         in
          evalExpr body (extendEnv ids args env)
```

4

Extendiendo el intérprete (proced.) 2

¿A qué valor debería evaluar un procedimiento? 3
Ejemplo:

```
let x = 5
 in let f = proc (y,z) +(y, -(z,x))
        x = 28
     in (f 2 x)
```
4

Observar: el cuerpo es evaluado 5

- solamente cuando es aplicado
- en un entorno que liga los parámetros formales a los argumentos de la aplicación

6

¿Y qué hay de las variables que ocurren libres en el procedimiento?

Evaluación de procedimientos

Ejemplo: 3

```
let x = 5
 in let f = proc (y,z) +(y, -(z,x))
        x = 28
     in (f 2 x)
```
4

Respecto a las variables que ocurren libres en el procedimiento 5

- deben respetar scoping léxico (o estático) 6
  - deben evaluarse usando el entorno en efecto en el momento de la creación 7
- Por lo tanto, debemos retener (junto con el cuerpo y los parámetros formales) las valores asociados a las variables libres en el momento en el que el procedimiento fue creado. 8

Clausuras

- cuerpo junto con un entorno y parámetros formales 3

```
data ExpressedValue = Nro {stripNro::Int} |
                      Closure [Symbol] Exp Env
```
4

- Volviendo a los valores expresados/denotados: 5
  - **Valores expresados = Números + Clausuras** 6
  - **Valores denotados = Números + Clausuras**

Extendiendo el intérprete

3

```
evalExpr (ProcExp ids body) env = Closure ids body env
evalExpr (AppExp rator rands) env =
        let proc =  evalExpr rator env
            args =  map (\x->evalExpr x env) rands
         in
          case proc of
            (Closure _ _ _) -> applyProcVal proc args
            _ -> error ("evalExpr: Intento de aplicar un
                         no-procedimiento "++show proc)

applyProcVal (Closure ids body env) args =
        evalExpr body (extendEnv ids args env)
```

Extendiendo el intérprete para SFL con recursión 2

- Ejemplos 3
- Sintaxis concreta y abstracta 4
- Implementando recursión a través de la creación de clausuras durante el lookup (V1) 5
  - Entornos en notación de sintaxis abstracta (anexando un nuevo tipo de rib) 6
  - Entornos en notación procedural
- Implementando recursión armando clausura una única vez (V2) 7
  - Entornos en notación de sintaxis abstracta (utilizando estructuras circulares) 8

Ejemplos en SFL con recursión 2

3

- ```
  letrec
    fact(x) = if zero?(x)
                then 1
                else *(x,(odd sub1(x)))
  in (fact 6)
  ```
- ```
  letrec
    even(x) = if zero?(x)
                then 1
                else (odd sub1(x))
    odd(x)  = if zero?(x)
                then 0
                else (even sub1(x))
  in (odd 13)
  ```

Sintaxis concreta y abstracta 3

Sintaxis concreta 4

```
<expr> ::=
    letrec
      {<identifier> ({<identifier>}*(,)) = <expr>}*
      in <expr>
```

5

Sintaxis abstracta 6

```
data Exp = LitExp Int |
           ... |
           LetRecExp [Symbol] [[Symbol]] [Exp] Exp
```

7

Implementación: versión ingenua 2

```
letrec 3
 even(x) = if zero?(x) then 1 else (odd sub1(x)) 4
 odd(x)  = if zero?(x) then 0 else (even sub1(x))
in (odd 13)
```

Manipulación ingenua de entornos y clausuras: 5

```
    /          / +-------+-------+                6
    |          | | var1  | val1  |
    |          | +-------+-------+
    | old-env  |   ...     ...
    |          | +-------+-------+
env |          | | varn  | valn  |
    |           +-------+-------+
    |            | even  | (Clausura ... old-env)
    |            +-------+-------+
    |            | odd   | (Clausura ... old-env)
                +-------+-------+
```

Clausuras deben referenciar **env** (y no **old-env**). Circularidad!

Implementación: Creación de clausuras al hacer lookup 2

- Idea: demorar creación de clausuras hasta que el procedimiento recursivo es llamado 3

```
    /          / +-------+-------+
    |          | | var1  | val1  |
    |          | +-------+-------+
    | old-env  |   ...     ...
    |          | +-------+-------+
env |          | | varn  | valn  |
    |           +-------+-------+
    |            | even  | al hacer lookup (Closure ... env)
    |            +-------+-------+
    |            | odd   | al hacer lookup (Closure ... env)
                +-------+-------+
```

4

Creación de clausuras al hacer Lookup - v1 2

- En ésta versión los entornos se representan como un tipo algebraico 3
- Se agrega un nuevo constructor para extender entornos con declaraciones recursivas locales

```
data Env = EmptyEnv |
           ExtendedEnvRecord [Symbol]
                             [ExpressedValue]
                             Env |
           RecursivelyExtendedEnvRecord [Symbol]
                                        [[Symbol]]
                                        [Exp]
                                        Env
```

4

Creación de clausuras al hacer Lookup - v1 2

- El código para `evalExpr` es el esperado 3

```
evalExpr (LetRecExp procNames idss bodies letrecBody) env =
       (evalExpr letrecBody
          (extendEnvRecursively procNames idss bodies env))

extendEnvRecursively procNames idss bodies env =
          RecursivelyExtendedEnvRecord procNames idss bodies env
```

4

- Lo interesante es cómo se realiza el lookup de un procedimiento recursivo en el entorno... 5

Creación de clausuras al hacer Lookup - v1 2

```
applyEnv::Env->Symbol->ExpressedValue
applyEnv EmptyEnv sym =
      error ("applyEnv: Sin binding para "++ show sym)
applyEnv (ExtendedEnvRecord syms vals env) sym
     = case findInList syms sym of
       (Just n) -> vals!!n
       _ -> applyEnv env sym
applyEnv env@(RecursivelyExtendedEnvRecord procNames idss bodies
     = case findInList procNames sym of
       (Just n) -> Closure (idss!!n) (bodies!!n) env
       _ -> applyEnv oldEnv sym
```

Creación de clausuras al hacer Lookup - v2 2

- Entornos representados como funciones 3

```
empty-env sym =
     error ("empty-env: Sin binding para "++ show sym)

extend-env syms vals env sym =
      let pos = findInList sym syms
       in case pos of
            (Just n) -> vals!!n
            _ -> apply-env env sym
```

4

```
apply-env env sym = env sym
```

5

Creación de clausuras al hacer Lookup - v2 2

- Agregamos una nueva operación para extender entornos 3

```
extend-env-recursively proc-names idss bodies old-env =
  let rec-env sym =
     let pos = findInList sym proc-names
       in case pos of
            (Just n) -> Closure idss!!pos bodies!!pos rec-env
            _ -> apply-env old-env sym
   in rec-env
```

4

Creación de clausuras al hacer Lookup - v2 2

- `evalExpr`: sin cambios respecto a v1 3

```
evalExpr (LetRecExp procNames idss bodies letrecBody) env =
       (evalExpr letrecBody
          (extendEnvRecursively procNames idss bodies env))

extendEnvRecursively procNames idss bodies env =
     RecursivelyExtendedEnvRecord procNames idss bodies env
```
4

Creación única de clausuras 3

- En los dos casos previos las clausuras se creaban a la hora de hacer lookup 4
- Esto no es necesario dado que el entorno de la clausura creada siempre es el mismo
- Utilizando estructuras circulares podemos limitarnos a construir clausuras una sola vez
- Asumimos representación estándar de entornos

```
data Env = EmptyEnv |
           ExtendedEnvRecord [Symbol]
                             [ExpressedValue]
                             Env
```

5

Implementación: Creación única de clausuras 2

- El evaluador de expresiones es el mismo que antes 3

```
evalExpr (LetRecExp procNames idss bodies letrecBody) env =
       (evalExpr letrecBody
          (extendEnvRecursively procNames idss bodies env))
```
4

- Pero las clausuras se crean al procesar la declaración 5

```
extendEnvRecursively procNames idss bodies oldEnv =
    let
      env = ExtendedEnvRecord procNames lst oldEnv
      lst = map (\(xs,y) -> Closure xs y env) (zip idss bodies)
     in  env
```
6